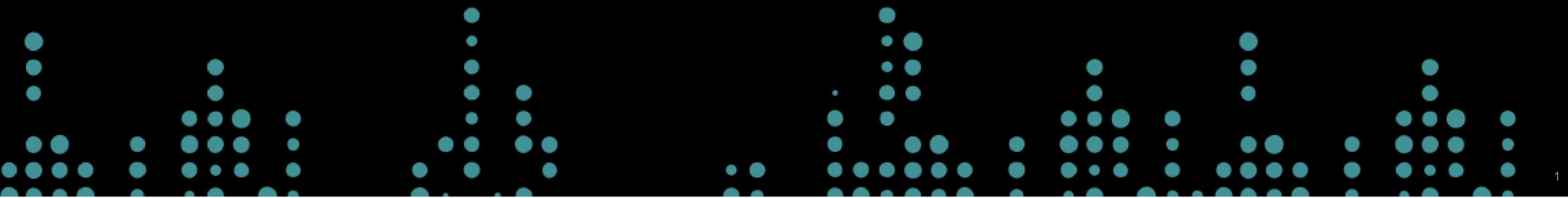


TEMAS AVANZADOS:

Medidas de similaridad y distancia

UNIDADES 2 y 3

Fundamentos de Ciencia de Datos 2026



MEDIDAS DE SIMILARIDAD ENTRE CADENAS

SIMILARIDAD DE JARO

La **similaridad de Jaro** entre dos cadenas s_1 y s_2 se calcula de la siguiente manera:

$$sim_j = \frac{1}{3} \left(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m - t}{m} \right)$$

$|s_1|$ es la longitud de la cadena s_1 .

m es el número de caracteres que coinciden.

t es la mitad del número de caracteres transpuestos, de los que coinciden.

SIMILARIDAD DE JARO

¿Cuándo dos caracteres se consideran coincidentes según Jaro?

Dos caracteres se consideran coincidentes **si son iguales y la distancia entre sus posiciones no supera:**

$$\left[\frac{\max(|s_1|, |s_2|)}{2} \right] - 1$$

SIMILARIDAD DE JARO-WINKLER

Winkler propone una modificación a la fórmula anterior en la que se aumenta el rating de similaridad si las cadenas comparadas comienzan con el mismo prefijo:

$$sim_{j-w} = sim_j + lp (1 - sim_j)$$

es la similaridad de Jaro entre las cadenas x y y .

sim_j es la cantidad de caracteres que coinciden **exactamente** al comienzo de la cadena, hasta un máximo de 4 caracteres.

p es un factor de escalado que indica cuánto se ajusta por la coincidencia en el prefijo, que no debería exceder $p = 0.25$ (valor estándar: $p = 0.1$).

p

p

LIBRERÍA RAPIDFUZZ

Rapid

Fuzz

RapidFuzz es una biblioteca de código abierto para *fuzzy string matching* y cálculo de similitud de cadenas, escrita principalmente en C++ con enlaces para Python.

Ofrece una amplia gama de métricas de similitud y variantes basadas en tokens o proporciones ponderadas.

IMPLEMENTACIÓN EN PYTHON

Del módulo **rapidfuzz.distance** importamos **Jaro** y **JaroWinkler**, que permiten calcular las correspondientes medidas de similaridad entre cadenas.

```
from rapidfuzz.distance import Jaro, JaroWinkler
```

Como ejemplo, calculemos ambas medidas para las siguientes cadenas:

A	R	G	E	N	T	I	N	A
A	R	G	E	N	T	E	A	N

IMPLEMENTACIÓN EN PYTHON

Cálculo de la similaridad de Jaro

```
# Calculamos similaridad de Jaro
sim_j = round(Jaro.similarity('ARGENTINA', 'ARGENTEAN'), 4)

print(sim_j)
```

0.8843

IMPLEMENTACIÓN EN PYTHON

Cálculo de la similaridad de Jaro-Winkler

```
# Calculamos similaridad de Jaro-Winkler
sim_jw = round(JaroWinkler.similarity('ARGENTINA', 'ARGENTEAN'), 4)

print(sim_jw)
```

0.9306

SIMILARIDAD DE JARO/JARO-WINKLER

Veamos otro ejemplo, ahora con estas dos cadenas:

Cadena 1: **MARIELA F.**

Cadena 2: **MARLENA F.**

 ¿Cuáles serían la similaridades de Jaro/Jaro-Winkler que se obtendrían en este ejemplo?

SIMILARIDAD DE JARO/JARO-WINKLER

Cálculo de la similaridad de Jaro

```
# Calculamos similaridad de Jaro
sim_j = round(Jaro.similarity('MARIELA F.', 'MARLENA F.'), 4)

print(sim_j)
```

0.8963

Cálculo de la similaridad de Jaro-Winkler

```
# Calculamos similaridad de Jaro-Winkler
sim_jw = round(JaroWinkler.similarity('MARIELA F.', 'MARLENA F.'), 4)

print(sim_jw)
```

0.9274

DISTANCIA DE LEVENSHTTEIN

Las **distancias de edición** son métricas que **cuentan la menor cantidad de operaciones de edición que son necesarias para convertir a una cadena en otra.**

La más básica de ellas es la **distancia de edición de Levenshtein** y está definida como el menor número de operaciones de **inserción, borrado o sustitución de un único carácter** que son necesarias para convertir la cadena s_1 en la cadena s_2 .

s_1

s_2

🤔 ¿Cuál sería la distancia de Levenshtein entre las cadenas **LEWENSTEIN** y **LEVENSHTTEIN**?

DISTANCIA DE LEVENSHTTEIN

Para calcular esta métrica en Python, importaremos **Levenshtein** del módulo **rapidfuzz.distance**:

```
from rapidfuzz.distance import Levenshtein

# Calculamos distancia de Levenshtein
dis_lev = Levenshtein.distance('LEWENSTEIN', 'LEVENSHTTEIN')

print(dis_lev)
```

SIMILARIDAD (NORMALIZADA) DE LEVENSHTTEIN

De manera análoga a las métricas de similaridad vistas anteriormente, se define una medida de similaridad normalizada basada en la distancia de Levenshtein de la siguiente forma:

$$sim_{lev} = 1 - \frac{dist_{lev}(s_1, s_2)}{\max(|s_1|, |s_2|)}$$

Se puede verificar que .

$$0 \leq sim_{lev} \leq 1$$

SIMILARIDAD (NORMALIZADA) DE LEVENSHTTEIN

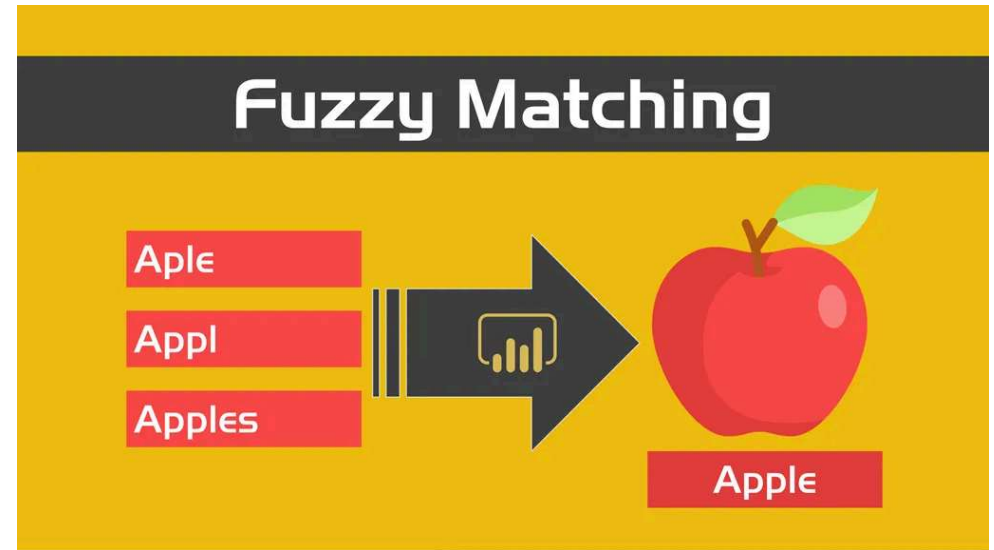
Podemos calcular esta nueva métrica con herramientas de **rapidfuzz** de la siguiente manera:

```
# Calculamos similaridad normalizada de Levenshtein
sim_lev = round(Levenshtein.normalized_similarity('LEWENSTEIN', 'LEVENSHTTEIN'))
print(sim_lev)
```

0.8182

FUZZY JOINS

El **enlace difuso** es una técnica utilizada para *matchear* datos que pueden no coincidir perfectamente debido a posibles inconsistencias, omisiones o errores.



En lugar de buscar coincidencias exactas, se buscan **coincidencias aproximadas** utilizando diversas herramientas, como las métricas de similaridad vistas anteriormente.

FUZZY JOINS - Ejemplo

Trabajamos en la Secretaría de Desarrollo Social de un municipio y necesitamos cruzar dos bases de datos:

- el **padrón electoral**, que contiene información de los vecinos registrados
- el **registro de beneficiarios de un programa de subsidios**.

Ambas bases fueron cargadas de forma independiente y en distintos momentos, por lo que los nombres no siempre coinciden exactamente. Nuestro objetivo es identificar qué beneficiarios del programa pueden ser vinculados con un registro en el padrón.

FUZZY JOINS - Ejemplo

df_padron: padrón electoral

```
df_padron = pd.DataFrame({'Nombre': ['María González', 'Carlos Alberto Rodríguez', 'Luisa Fernanda Martínez', 'Roberto Sánchez', 'Ana Laura Pérez'],  
    'DNI': [28456123, 31782456, 25963147, 33841290, 29571834],  
    'Localidad': ['Rosario', 'Rosario', 'Villa Gobernador Gálvez', 'Rosario', 'Funes']})  
  
print(df_padron)
```

	Nombre	DNI	Localidad
0	María González	28456123	Rosario
1	Carlos Alberto Rodríguez	31782456	Rosario
2	Luisa Fernanda Martínez	25963147	Villa Gobernador Gálvez
3	Roberto Sánchez	33841290	Rosario
4	Ana Laura Pérez	29571834	Funes

FUZZY JOINS - Ejemplo

df_beneficiarios: beneficiarios del programa de subsidios

```
df_beneficiarios = pd.DataFrame({'Nombre': ['Maria Gonzalez', 'Carlos Rodr',  
'Subsidio': [15000, 22000, 18000, 12000, 9000],  
'Fecha_Alta': ['2023-03-01', '2023-05-14', '2022-11-30', '2024-01-08', '20  
  
print(df_beneficiarios)
```

	Nombre	Subsidio	Fecha_Alta
0	Maria Gonzalez	15000	2023-03-01
1	Carlos Rodríguez	22000	2023-05-14
2	L. F. Martínez	18000	2022-11-30
3	Roberto Sanches	12000	2024-01-08
4	Jorge Medina	9000	2023-07-22

FUZZY JOINS - Ejemplo

 Hasta donde sabemos, ambos datasets contienen información sobre personas que podrían coincidir. ¿Qué campo podría usarse para intentar vincularlos?

FUZZY JOINS - Ejemplo

🤔 Hasta donde sabemos, ambos datasets contienen información sobre personas que podrían coincidir. ¿Qué campo podría usarse para intentar vincularlos?

🤔 ¿Si intentáramos combinar ambos DataFrames usando `merge()` con la columna **Nombre** como clave, ¿cuántas filas esperaríamos que tenga el resultado? ¿Por qué?

FUZZY JOINS

Para realizar el enlace difuso, trabajaremos con dos componentes de la librería **rapidfuzz**:

- **fuzz**: provee las funciones de similaridad entre cadenas.
- **process**: permite buscar la mejor coincidencia entre una cadena y una lista de candidatos, aplicando la función de similaridad elegida.

El desarrollo completo del ejemplo está disponible en una notebook en la sección **Métodos Avanzados** del book 📖



[Abrir notebook](#)

fuzz.token_sort_ratio()

Antes de calcular la similaridad, esta función aplica un preprocesamiento a ambas cadenas. El comportamiento depende del parámetro **processor**:

1. Tokenización: divide cada cadena en *tokens*, es decir, en unidades individuales separadas por espacios. Por ejemplo, "Juan Pérez" se divide en los tokens ["Juan", "Pérez"].

2. Normalización: convierte todos los *tokens* a minúsculas y elimina signos de puntuación.

 **Esto último sólo ocurre si se especifica `processor = utils.default_process`.**

fuzz.token_sort_ratio()

3. Ordenamiento alfabético: los *tokens* se reordenan alfabéticamente y se vuelven a unir en una sola cadena.

4. Cálculo de similaridad: se calcula una métrica de similaridad entre las dos cadenas resultantes basada en la **distancia de edición de Levenshtein**, normalizada para devolver un valor entre 0 y 100.

DISTANCIAS Y MEDIDAS DE SIMILARIDAD EN DATOS NUMÉRICOS

MEDIR LA SIMILARIDAD ENTRE DATOS

En el análisis de datos, muchas veces necesitamos cuantificar qué tan parecidos o distintos son los objetos entre sí.

Esto es útil en varias tareas:

- **Clustering:** agrupar elementos en grupos homogéneos (*clusters*) en función de las similitudes entre ellos.
- **Detección de *outliers*:** identificar elementos muy diferentes al resto.
- **Clasificación de vecinos más cercanos (KNN):** clasificar elementos basándose en las características de otros elementos similares (*vecinos*).

DISTANCIA EUCLÍDEA

La **distancia euclídea** es la medida de distancia más popular.

Si $\mathbf{i}$ y $\mathbf{j}$ representan **dos observaciones (o elementos)** descritas por variables cuantitativas, la distancia euclídea entre ellas se define como:

$$d_E(\mathbf{i}, \mathbf{j}) = \sqrt{(x_{1i} - x_{1j})^2 + (x_{2i} - x_{2j})^2 + \dots + (x_{pi} - x_{pj})^2}$$

DISTANCIA EUCLÍDEA



IMPORTANTE

Esta medida es sensible a las escalas de las variables. Si las variables tienen rangos muy diferentes, las de mayor rango dominarán el cálculo de la distancia, minimizando el aporte de las demás. Por eso, cuando las escalas son diferentes, conviene **normalizar o estandarizar** los datos previamente (algo que veremos en la Unidad 5).

DISTANCIA EUCLÍDEA - Ejemplo

Supongamos que tenemos el siguiente DataFrame:

```
df = pd.DataFrame({'id': ['A', 'B', 'C', 'D', 'E'],  
                  'x1': [2.0, 2.5, 3.0, 7.5, 8.0],  
                  'x2': [3.0, 3.5, 3.2, 7.0, 7.8],  
                  'x3': [1.0, 1.2, 0.8, 6.5, 7.0]  
})  
  
print(df)
```

	id	x1	x2	x3
0	A	2.0	3.0	1.0
1	B	2.5	3.5	1.2
2	C	3.0	3.2	0.8
3	D	7.5	7.0	6.5
4	E	8.0	7.8	7.0

Antes de calcular distancias, verificamos si es necesario estandarizar los datos:

```
print(df.describe())
```

	x1	x2	x3
count	5.000000	5.000000	5.000000
mean	4.600000	4.900000	3.300000
std	2.902585	2.306513	3.157531
min	2.000000	3.000000	0.800000
25%	2.500000	3.200000	1.000000
50%	3.000000	3.500000	1.200000
75%	7.500000	7.000000	6.500000
max	8.000000	7.800000	7.000000

Las tres variables presentan rangos aproximadamente similares, por lo que ninguna dominará el cálculo de la distancia euclídea. En este caso, **no es necesario estandarizar previamente**.



La librería **SciPy** ofrece herramientas para calcular y representar **matrices de distancias**. Utilizaremos dos funciones del módulo **scipy.spatial.distance**:

- **pdist()**: *pairwise distances*, calcula todas las distancias por pares entre filas de un DataFrame o matriz. Para n observaciones, devuelve un vector con las distancias únicas.
- **squareform()**: convierte ese vector en una **matriz cuadrada simétrica** de .

$$n \times n$$

```

from scipy.spatial.distance import pdist, squareform

# Matriz de distancias euclídeas
dist_matrix = pd.DataFrame(squareform(pdist(df[['x1', 'x2', 'x3']], metric='euclidean'),
                                   index = df['id'], columns = df['id']))

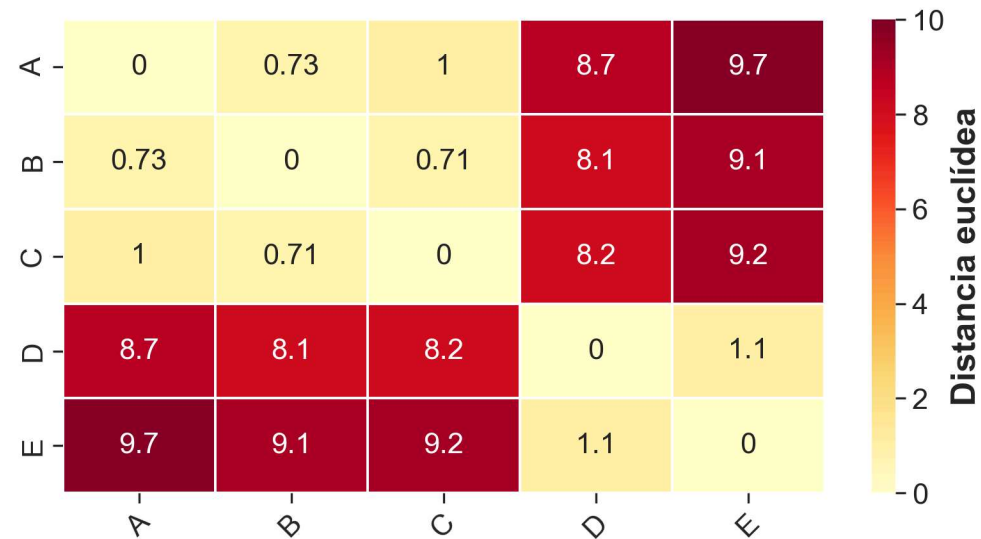
dist_matrix.round(3)

```

id	A	B	C	D	E
id					
A	0.000	0.735	1.039	8.746	9.749
B	0.735	0.000	0.707	8.083	9.076
C	1.039	0.707	0.000	8.196	9.198
D	8.746	8.083	8.196	0.000	1.068
E	9.749	9.076	9.198	1.068	0.000

Representación visual de la matriz de distancias:

```
sns.heatmap(dist_matrix, annot = True, vmin = 0, vmax = 10, cmap='YlOrRd')
```



🧐 ¿Qué interpretación podemos hacer de las distancias determinadas?

DISTANCIA EUCLÍDEA - Ejemplo

El heatmap revela una estructura clara en los datos: existen **dos grupos bien diferenciados**.

- Las observaciones **A, B y C** son muy similares entre sí (distancias pequeñas, celdas claras).
- Las observaciones **D y E** también son muy similares entre sí.
- En cambio, la distancia entre cualquier elemento de {A, B, C} y cualquier elemento de {D, E} es considerablemente mayor (celdas oscuras).

DISTANCIA MANHATTAN



También llamada **City-Block** o **Taxicab**, la **distancia Manhattan** mide la distancia entre dos puntos considerando que **sólo se puede avanzar en líneas rectas horizontales o verticales**, como si uno se desplazara por las calles de una ciudad en forma de cuadrícula.

Para dos objetos $\mathbf{i}$ y $\mathbf{j}$ con p atributos se define como:

$\mathbf{i}$ $\mathbf{j}$ p

$$d_{MHT}(\mathbf{i}, \mathbf{j}) = |x_{1i} - x_{1j}| + |x_{2i} - x_{2j}| + \dots + |x_{pi} - x_{pj}|$$

DISTANCIA MANHATTAN

```
# Matriz de distancias Manhattan
dist_matrix_manhattan = pd.DataFrame(squareform(pdist(df[['x1', 'x2', 'x3']
    index = df['id'],
    columns = df['id']))

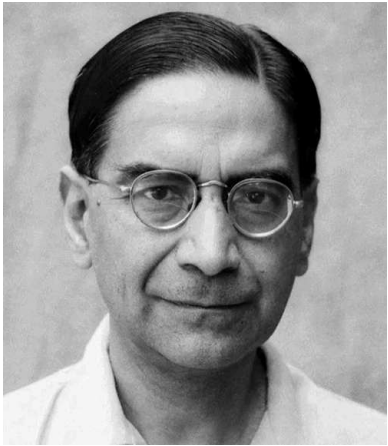
dist_matrix_manhattan.round(3)
```

id	A	B	C	D	E
id					
A	0.0	1.2	1.4	15.0	16.8
B	1.2	0.0	1.2	13.8	15.6
C	1.4	1.2	0.0	14.0	15.8
D	15.0	13.8	14.0	0.0	1.8
E	16.8	15.6	15.8	1.8	0.0

MANHATTAN vs. EUCLÍDEA

- En espacios de alta dimensionalidad, la distancia Manhattan puede comportarse mejor que la euclídea, ya que esta última tiende a perder capacidad discriminatoria a medida que aumenta el número de dimensiones (*curse of dimensionality*).
- La distancia Manhattan es preferible si los datos están ordenados como grilla o cuadrícula, donde el movimiento diagonal no tiene sentido.
- La distancia Manhattan suele ser más adecuada cuando los atributos contribuyen de manera aditiva e independiente, ya que mide el desplazamiento total coordenada por coordenada.

DISTANCIA DE MAHALANOBIS



Fue propuesta por Prasanta Chandra Mahalanobis en 1936. Mide qué tan lejos está una observación respecto del centro de una distribución multivariada, usualmente representado por el vector de medias de la distribución (), teniendo en cuenta la **dispersión de los datos** y la **correlación entre las variables**.

A diferencia de la distancia euclídea, **no trata a todas las direcciones por igual**: penaliza más aquellas que son poco compatibles con la forma de la nube de datos. Se usa mucho en clasificación y detección de valores atípicos multivariados.

La distancia euclídea puede escribirse en forma matricial como:

$$d_E(\mathbf{r}_1, \mathbf{r}_2) = \sqrt{(\mathbf{r}_1 - \mathbf{r}_2)^T \mathbf{I} (\mathbf{r}_1 - \mathbf{r}_2)}$$

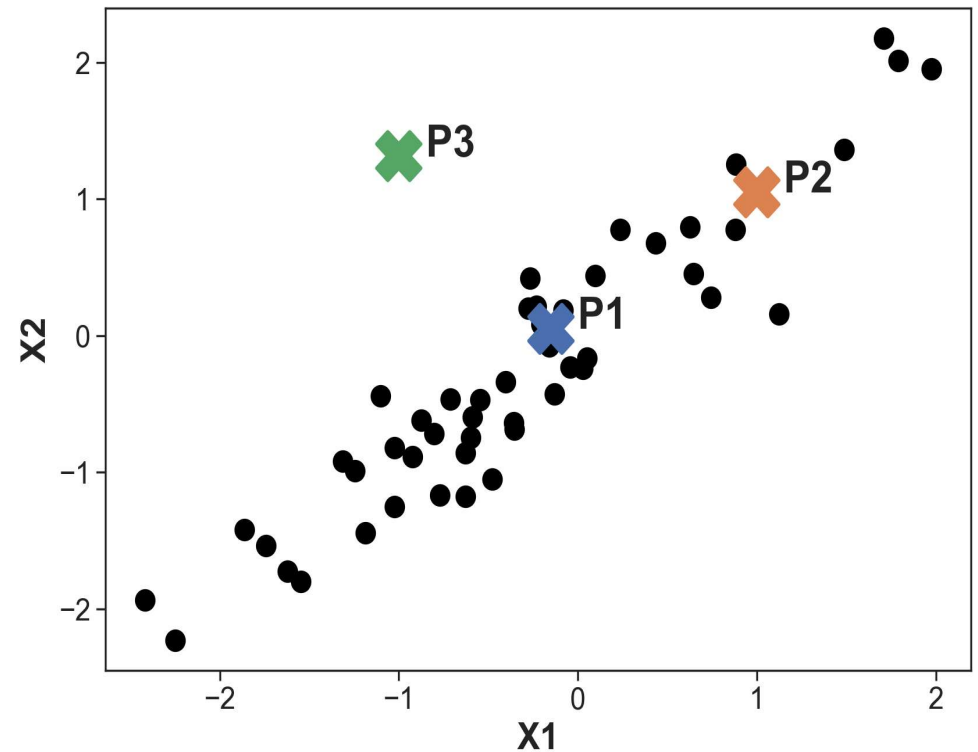
donde $\mathbf{I}$ es la matriz identidad.

La distancia de Mahalanobis reemplaza $\mathbf{I}$ por la inversa de la matriz de covarianza : Σ^{-1}

$$d_M(\mathbf{r}_1, \mathbf{r}_2) = \sqrt{(\mathbf{r}_1 - \mathbf{r}_2)^T \Sigma^{-1} (\mathbf{r}_1 - \mathbf{r}_2)}$$

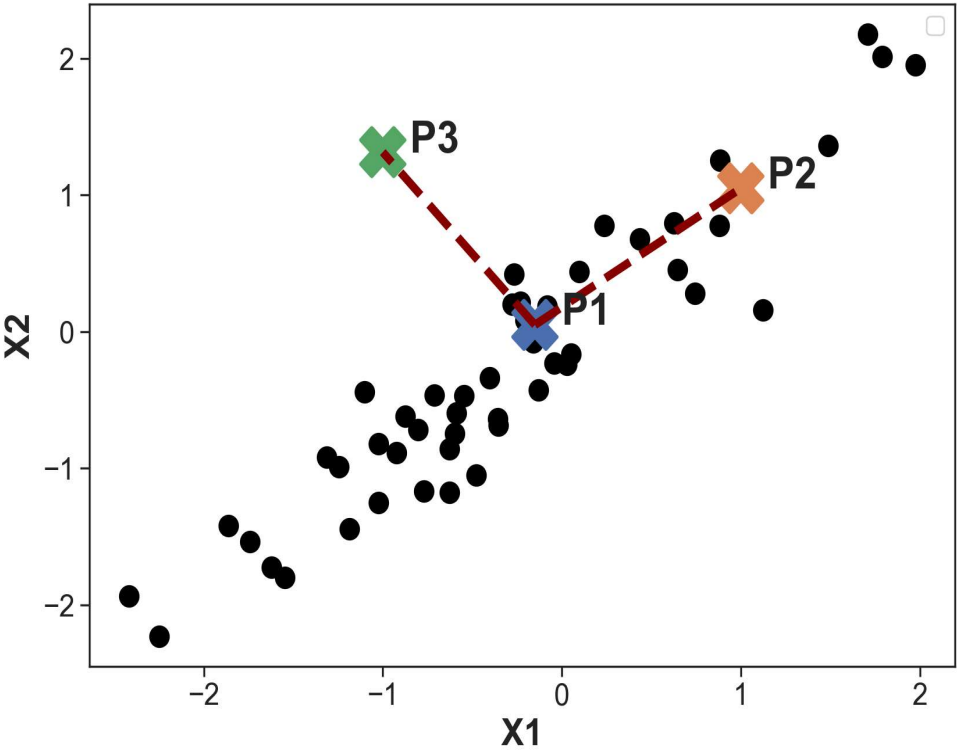
Al incorporar Σ^{-1} , la distancia se ajusta automáticamente por la variabilidad de cada variable y la correlación entre variables.

Supongamos que contamos con datos de dos variables con una fuerte correlación lineal positiva, y consideremos tres puntos: ,
(dentro de la nube) y *(fuera de la nube)*.
 P_1 P_2
 P_3



Desde el punto de vista de la **distancia euclídea**, y se encuentran prácticamente a la misma distancia de :

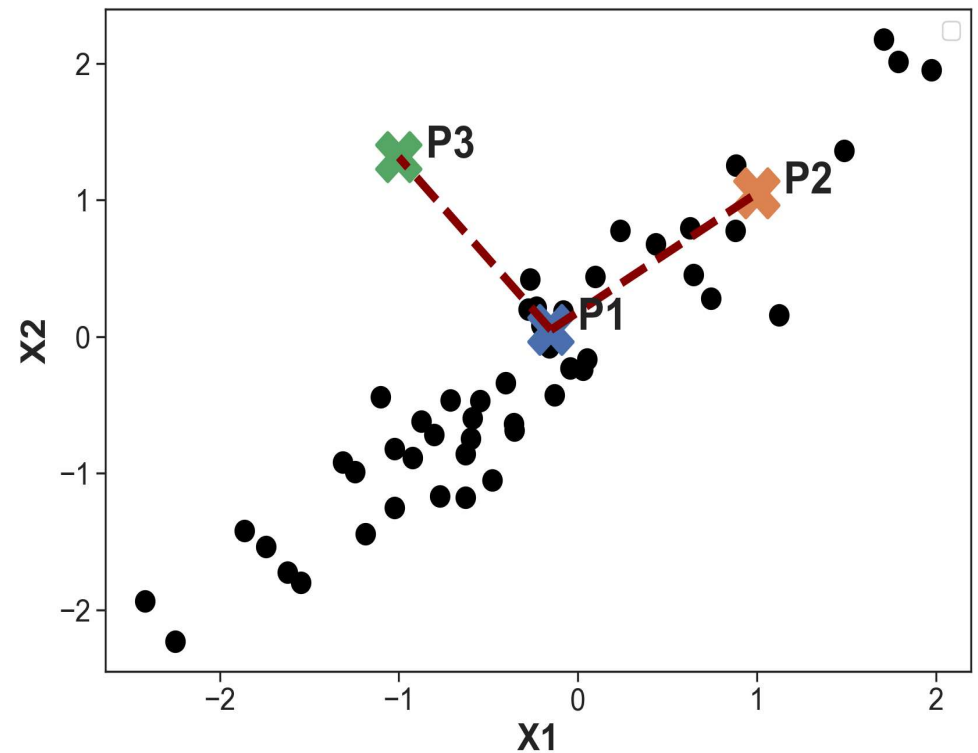
	P1	P2	P3
P1	0.000	1.524	1.524
P2	1.524	0.000	2.017
P3	1.524	2.017	0.000



😬 Sin embargo, observando la forma de la nube de datos, resulta razonable considerar que **está más “alineado” con la estructura de los datos que** P_2 .

Aunque ambos puntos estén a una distancia euclídea similar de P_3 , **sigue la dirección principal de variación de la nube** P_1 , mientras que P_2 se aparta de ella.

P_3



La distancia de Mahalanobis **ajusta las distancias usando la matriz de covarianza de los datos**. Las direcciones de mayor variabilidad en los datos “pesan menos”, las direcciones poco compatibles con la nube “pesan más”.

Matriz de distancias de Mahalanobis:

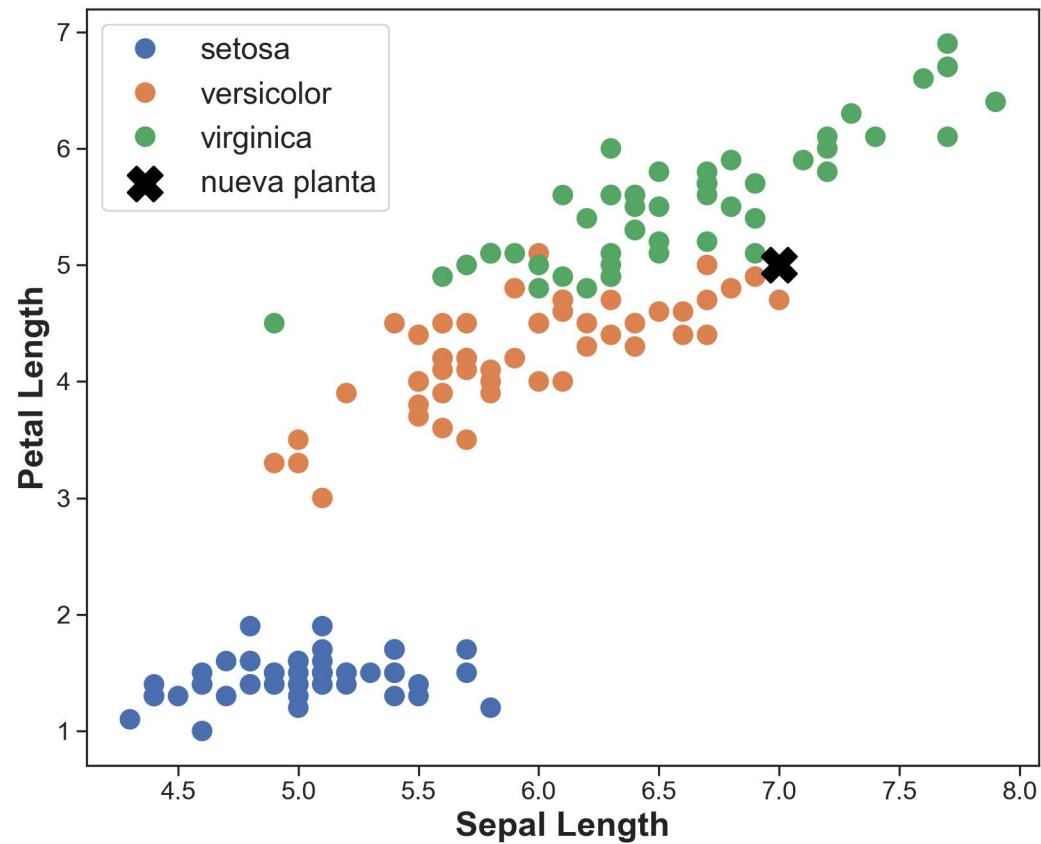
	P1	P2	P3
P1	0.000	1.178	6.083
P2	1.178	0.000	6.625
P3	6.083	6.625	0.000

Ahora : Mahalanobis reconoce que P_1 es más compatible con la estructura de los datos $d_M(P_1, P_3) < d_M(P_1, P_2)$

DISTANCIA DE MAHALANOBIS - Ejemplo *Iris*

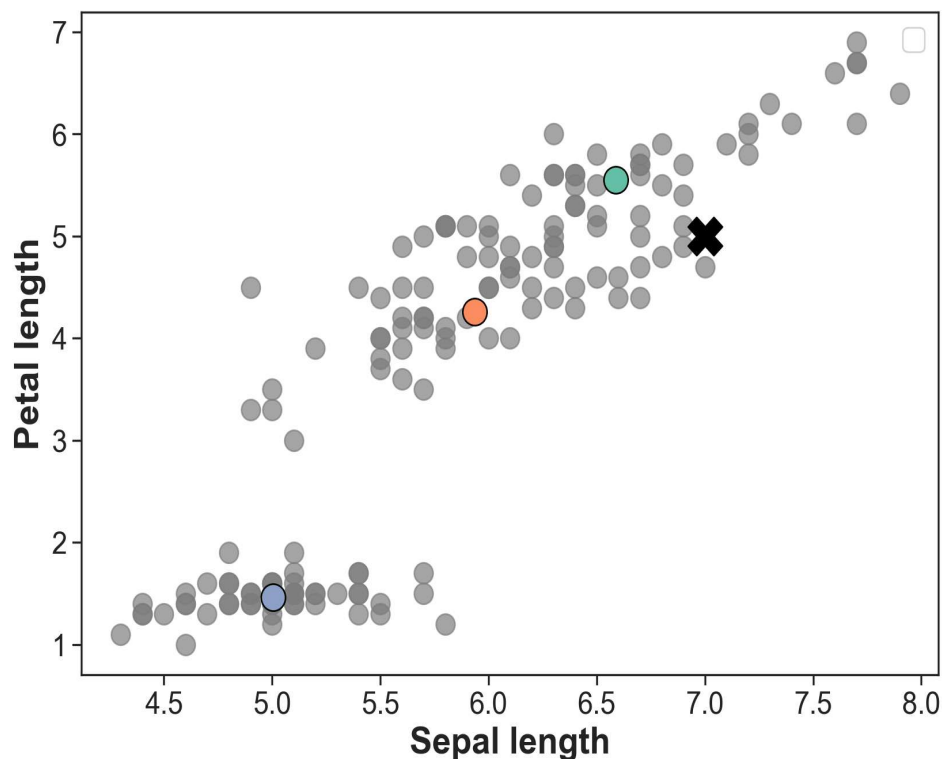
Vamos a usar el dataset **Iris**. Para simplificar visualización y cálculo, consideraremos solo dos variables: **sepal_length** y **petal_length**.

Objetivo: dada una nueva planta con **sepal_length** = 7 y **petal_length** = 5, determinar a qué especie pertenece.



🤔 Intuitivamente, ¿a qué especie pertenecería esta nueva planta?

DISTANCIA DE MAHALANOBIS - Ejemplo *Iris*



Vamos a responder esta pregunta de forma más rigurosa calculando la **distancia de Mahalanobis de la nueva planta al *centroide* (vector de medias) de cada especie**, usando la matriz de covarianza muestral de cada especie como distribución de referencia.

DISTANCIA DE MAHALANOBIS - Ejemplo *Iris*

	species	dist_mahal_cent
0	setosa	20.374
1	versicolor	2.062
2	virginica	3.168

La nueva planta se clasificaría como **versicolor**, por ser la especie cuyo centroide presenta la menor distancia de Mahalanobis al nuevo punto.

DISTANCIA DE MAHALANOBIS - Ventajas

- **Considera la correlación:** incorpora la estructura de covarianza entre variables, ajustando las distancias según la forma y orientación de la “nube de datos”.
- **Es independiente de la escala:** no requiere estandarizar previamente las variables. La multiplicación por S^{-1} incorpora automáticamente la variabilidad de cada variable, dándole a cada una un peso proporcional a su dispersión en los datos.
- **Permite la detección de valores atípicos multivariados:** observaciones alejadas de la estructura general de los datos presentan distancias de Mahalanobis elevadas respecto al centroide de su grupo.

SIMILARIDAD DE COSENO

Antes de introducir esta medida, necesitamos entender cómo se representan los documentos de texto de forma numérica.

REPRESENTACIÓN DE TEXTO: BoW

El modelo **Bag of Words** (BoW) representa cada documento como un vector de frecuencias de términos (***term-frequency vector***): cada dimensión corresponde a una palabra del vocabulario y su valor indica cuántas veces aparece esa palabra en el documento.

Ejemplo:

	"ciencia"	"datos"	"modelo"	"error"
doc1	2	3	1	0
doc2	1	2	0	4
doc3	2	3	1	0

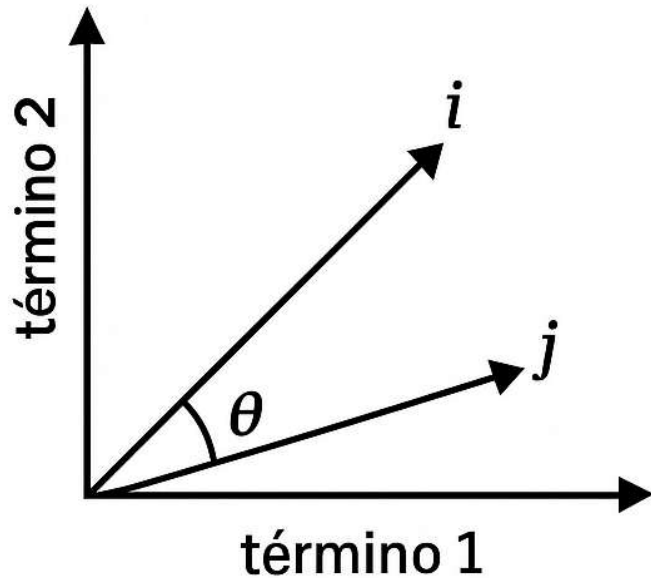
⚠ En la práctica, el vocabulario puede tener miles de palabras, por lo que estos vectores suelen ser muy largos y con muchos ceros (*vectores dispersos* o *sparse*).

¿Por qué no usar distancias tradicionales para comparar documentos? Consideremos estos tres documentos representados con BoW:

	“modelo”	“embeddings”
doc1	10	2
doc2	150	7
doc3	2	18

doc1 y **doc2** hablan principalmente de “modelo”. Sin embargo, **doc2** es mucho más largo. La distancia euclídea entre **doc1** y **doc2** será grande por esa diferencia de escala, aunque ambos parecen ser similares en temática **por las proporciones en que aparecen los términos.**

SIMILARIDAD DE COSENO



La **similaridad de coseno** resuelve esto: en lugar de medir la distancia entre los extremos de los vectores, mide el **ángulo entre ellos** (θ), lo que la hace invariante a la longitud del documento.

SIMILARIDAD DE COSENO

Para dos vectores $\mathbf{i}$ y $\mathbf{j}$, define como:

$\mathbf{i}$ $\mathbf{j}$

$$sim_{COS}(\mathbf{i}, \mathbf{j}) = \frac{\mathbf{i} \cdot \mathbf{j}}{||\mathbf{i}|| ||\mathbf{j}||}$$

En la expresión anterior, $||\mathbf{w}||$ representa la **norma Euclídea** del vector $\mathbf{w}$, es decir: .

El cociente divide el producto punto por el producto de las normas, lo que equivale a **normalizar ambos vectores** antes de compararlos. Por eso el resultado no depende de la longitud de los documentos.

SIMILARIDAD DE COSENO

Si la $\cos = 1$, los vectores apuntan en la misma dirección: se trata de documentos muy similares en composición temática.

Si $\cos = 0$, los vectores son ortogonales: los documentos no comparten ningún término en común.

⚠ En el contexto de **vectores de frecuencias de términos**, todos los valores son no negativos, por lo que $\cos \in [0, 1]$. En otros contextos con valores negativos, la similaridad puede tomar valores en $[-1, 1]$.

SIMILARIDAD DE COSENO - Ejemplo

Volvamos al ejemplo anterior. Calculemos primero la distancia euclídea entre los documentos **doc1**, **doc2** y **doc3**:

id	doc1	doc2	doc3
id			
doc1	0.000	140.089	17.889
doc2	140.089	0.000	148.408
doc3	17.889	148.408	0.000

SIMILARIDAD DE COSENO - Ejemplo

Calculemos ahora la similaridad de coseno entre **doc1** y **doc2**, que comparten temática (“modelo”) pero difieren en longitud:

Similaridades de coseno entre los distintos pares de documentos:

```
sim(doc1, doc2): 0.989
```

```
sim(doc1, doc3): 0.303
```

```
sim(doc2, doc3): 0.157
```

 ¿Qué interpretación hacemos de estos resultados?
¿Coincide con lo que esperaríamos intuitivamente?

SIMILARIDAD DE COSENO - Ejemplo 2

Ejemplo extraído de *Data Mining: Concepts and Techniques* (Han, Kamber & Pei, 2012). Se cuenta con 5 documentos con los siguientes vectores de frecuencias de términos:

Document	team	coach	hockey	baseball	soccer	penalty	score	win	loss	season
<i>Document1</i>	5	0	3	0	2	0	0	2	0	0
<i>Document2</i>	3	0	2	0	1	1	0	1	0	1
<i>Document3</i>	0	7	0	2	1	0	0	3	0	0
<i>Document4</i>	0	1	0	0	1	2	2	0	3	0

Supongamos que $\mathbf{v}_i$ y $\mathbf{v}_j$ son los dos primeros vectores de frecuencia de términos en la tabla anterior. ¿Cuán similares son $\mathbf{v}_i$ y $\mathbf{v}_j$?

SIMILARIDAD DE COSENO - Ejemplo 2

```
from scipy.spatial.distance import cosine

i = [5,0,3,0,2,0,0,2,0,0]
j = [3,0,2,0,1,1,0,1,0,1]

# La función cosine() devuelve la distancia coseno,
# por eso le restamos a 1 para obtener la correspondiente medida de similitud
cos_sim = 1 - cosine(i, j)
print(cos_sim.round(3))
```

0.936

Usando la similaridad de coseno para comparar estos dos documentos, los mismos serían considerados muy similares entre sí en cuanto a la temática: hay gran coincidencia de términos.